

ФИНАЛЬНЫЙ ПРОЕКТ, NFACTORIAL LLM ENGINEERING

RemarkRound

Приемка веб-проекта: система сверяет замечания заказчика с ТЗ,
решает человек

remark-round.up.railway.app

Алмас Доненбетов, сентябрь 2026

ПРОБЛЕМА

В журнале приемки поломки, желания и пробелы ТЗ лежат в одной колонке

1

Поломка

"Кнопка 'Сохранить' серая"

В ТЗ §2.1 основная кнопка синяя.
Это работа по договору.

2

Новое желание

"Хотим темную тему"

В ТЗ этого нет. Это отдельная
работа и отдельные деньги.

3

Пробел в ТЗ

"Нет выгрузки в Excel"

Документы молчат. Решить
может только человек.

В спешке все это уходит разработчикам как баги. Спор с заказчиком идет по памяти и в почте.

Система готовит разбор. Решает человек

1

Замечание

Строка журнала и
скриншот от заказчика

2

Опора

Поиск по ТЗ и
протоколам, цитата с
номером раздела

3

Разбор

Факты кадра,
предложенный класс,
черновик для РМ

4

Решение

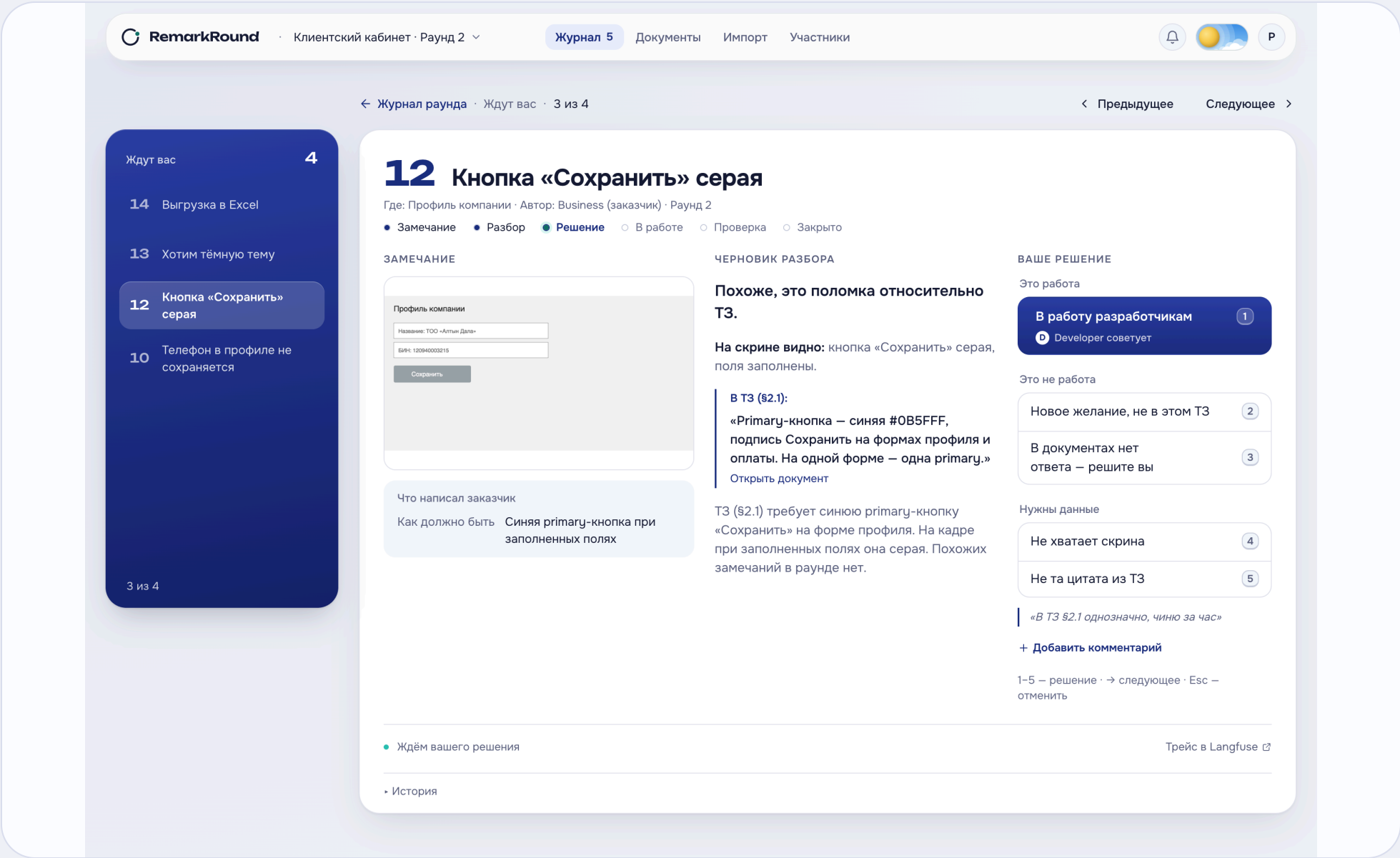
Одна из пяти кнопок
РМ. Только после нее
замечание становится
работой

На ретесте пиксели сравнивает алгоритм, модель
поясняет изменение, закрывает замечание только
заказчик.

**Jira хранит работу. RemarkRound помогает РМ решить,
работа ли это.**

Один цикл под тремя ролями

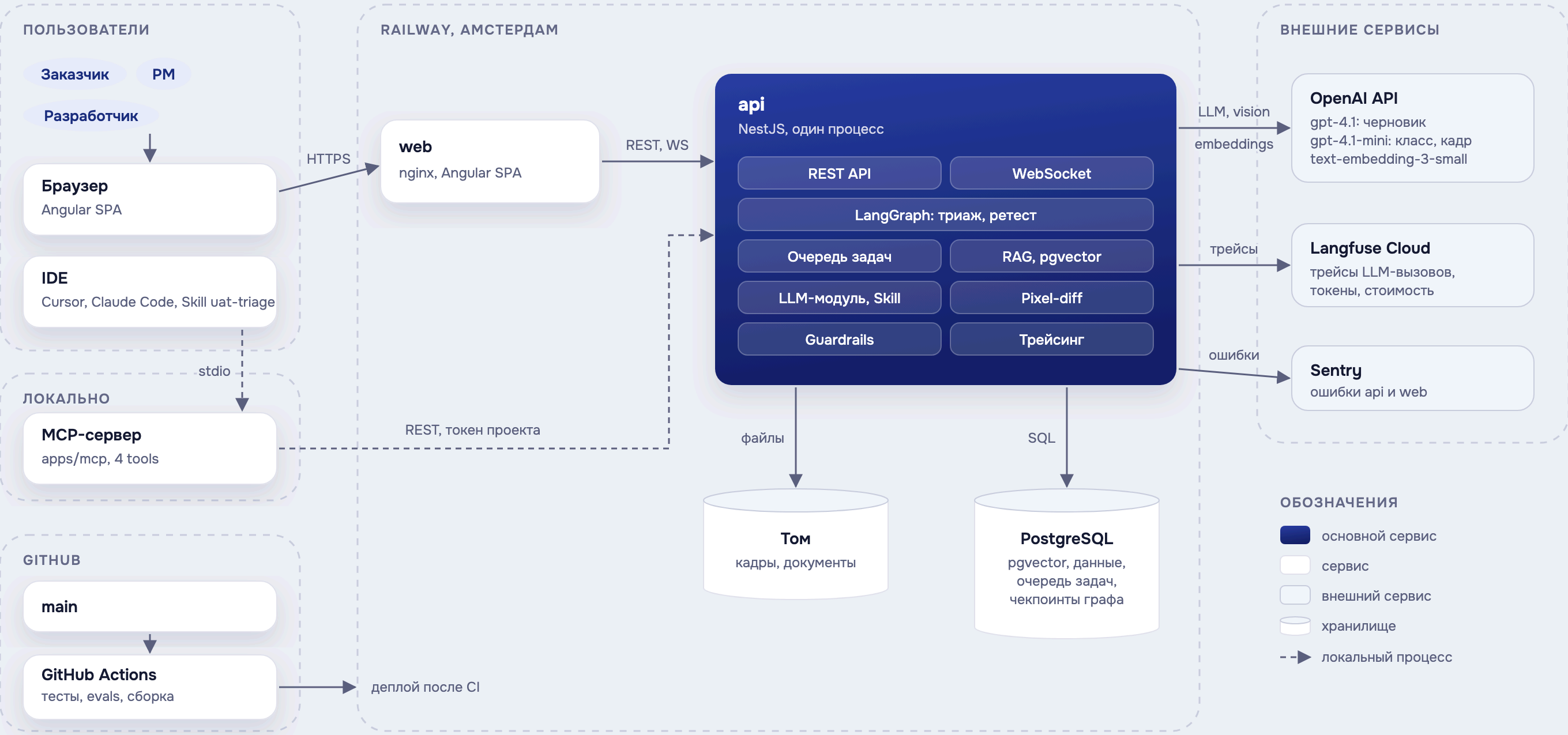
- Заказчик добавляет замечание
- РМ видит цитату из ТЗ и решает кнопкой
- Разработчик получает только принятые поломки
- Заказчик прикладывает кадр "стало" и закрывает
- Трейс разбора в Langfuse



Один процесс API, одна база, внешние сервисы по API

RemarkRound: архитектура продакшена

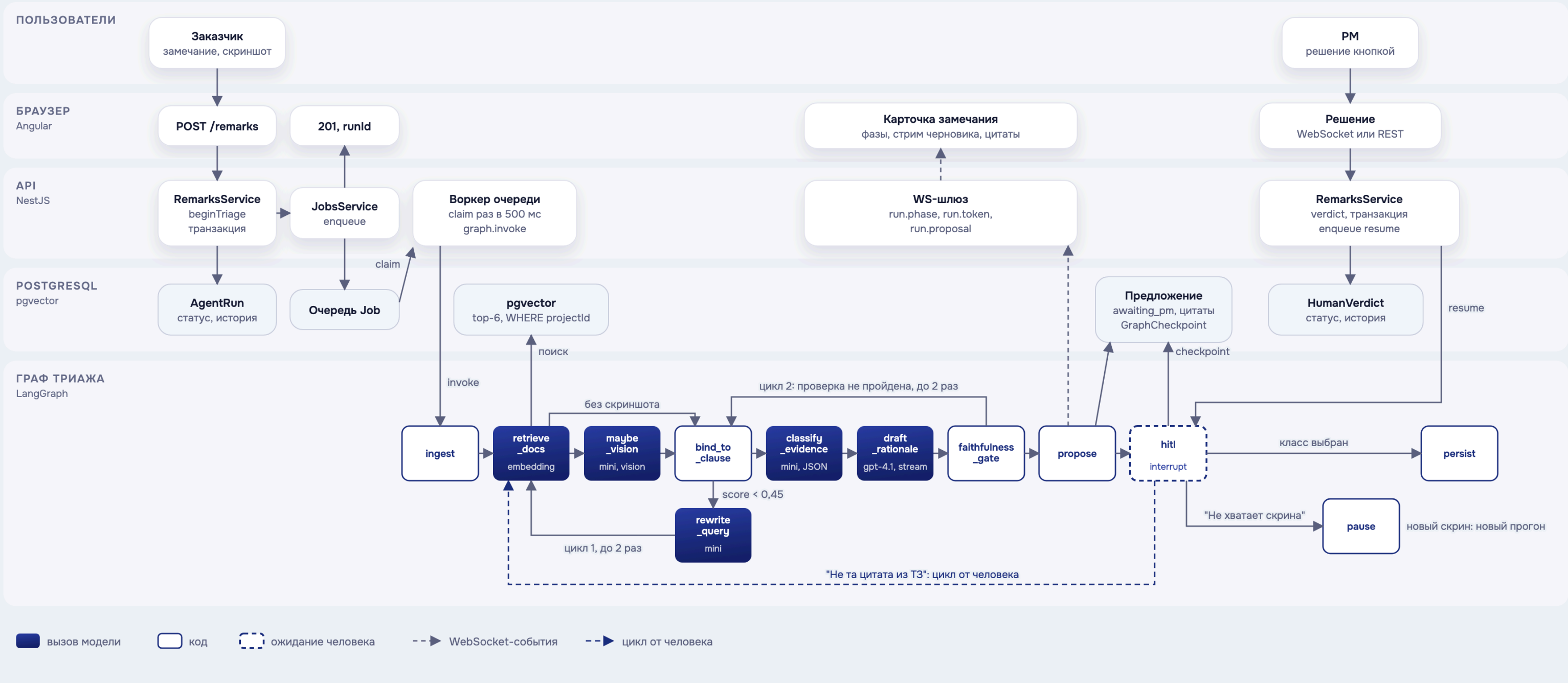
Сервисы и потоки данных, сентябрь 2026



Путь замечания через граф триажа

Путь замечания: от журнала до решения РМ

Граф триажа на LangGraph, время слева направо



Почему LangGraph

Циклы с лимитом

Переписать запрос: до 2 раз.
Переделать черновик: до 2 раз.
Третий провал: "недостаточно данных" ставит код.

Пауза на человека

interrupt: граф спит часы и дни.
Состояние в Postgres, resume берет воркер очереди.
"Не та цитата из ТЗ" продолжает тот же прогон.

Одно хранилище

Чекпоинты, очередь и векторы в одной Postgres.
Свой чекпоинтер на Prisma, около 150 строк.

12

нод и 4 условных перехода в графе триажа

2 + 1

цикла: два с лимитом, один от человека

9

вызовов модели максимум за проход

1 = 1

один прогон, один трейс

CrewAI и Parlant: Python и второй сервис. Сравнение по документации, прототипов не делали (ADR 014).

Цитата равна разделу Т3

- Чанк: раздел по заголовкам, длинные разделы окнами 220 слов с перекрытием 40
- Эмбединги: text-embedding-3-small, 1536 измерений
- Хранилище: pgvector, индекс HNSW, фильтр проекта в SQL
- Порог опоры 0,45. Ниже порога запрос переписывается, до 2 раз
- Reranker не ставили: корпус мал, опоры выбирает шаг classify

Мультимодальность

3 из 3 исправлений узнает ретест с моделью, без нее 0 из 3

15 из 16 прогонов верно в кейсе "текст врет" с картинкой, 0 из 1 без нее

Документы: PDF, DOCX, DOC → разделы.
Журнал XLSX с картинками

16/25

hit@1

20/25

hit@3

23/25

hit@6

Живой поиск на golden, 20.09.
Корпус 11 фрагментов

На Т3 в 38 разделов поиск не мерили. Чанкинг, top-1 из 8 запросов: по заголовкам 6-7, окна 4-5.

Golden 49: три метрики и метрика-ворота

Система	Опора	Метрика-ворота: честность
Константа "всегда не знаю"	20/33	33/33
Правила без модели	17/33	33/33
gpt-4.1-mini + gpt-4.1	28-29/33	33/33

Поиск hit@1/3/6: 16/20/23 из 25. Ретест: 14 из 15, одно ложное "исправлено".

33 разбора, 15 ретестов, 1 утечка между проектами, 12 типов кейсов. CI гоняет офлайн-evals на каждый push, порог опоры 0,65.

Чего метрики не видят

- Честность: та же проверка стоит в графе, поэтому 33/33 у всех
- Качество текста черновика не измеряется
- Golden синтетический, писали сами
- Разброс двух одинаковых прогонов: 1 кейс

Гиперпараметры и модели: 20 прогонов, \$1,86

Параметр	Опора из 33
По умолчанию, два прогона	28 и 29
classify, T 0,3 и 0,7	30 и 30
черновик, T 0 и 0,7	28 и 29
top_p 0,8	29
max_tokens 120 и 400	30 и 29

Параметры в пределах шума, дефолты оставлены: classify T 0, черновик T 0,3, top_p не передаем. Температуру черновика метрики не видят.

Модели	Опора	\$	p95
mini + 4.1	28-29	0,0033	5,7-5,9 с
4.1 + 4.1	27	0,0072	8,2 с
nano + 4.1	22	0,0024	4,9 с
mini + mini	29	0,0017	5,2 с

Черновик на mini вдвое дешевле, метрики те же. В 4 текстах из 33 фактические ошибки. Остались на gpt-4.1.

А/В

Ретест: выигрывает предпроверка кадров

Вариант, 15 кейсов	Верно	Ложных "исправлено"	Исправлений узнано	\$ за ретест
Константа "сравнить нельзя"	11	0	0 из 3	0
H0: два кадра в модель	11-12	3	3 из 3	0,0010
H1: pixel-diff и пояснение , после правки 21.09	14	1	3 из 3	0,0007
H0 с предпроверкой, расчет	14	1	3 из 3	0,0005

Без модели ретест не узнает ни одного исправления. Качество сверх этого дает предпроверка кадров алгоритмом, картинка диффа нужна человеку. H1 остался в продукте за предпроверку и объяснимость.

ВЫВОДЫ

Гипотезы, которые не подтвердились

Картинка диффа в модели улучшает ретест	Выигрыш дает предпроверка кадров. Сама модель на кадрах нужна: без нее 0 из 3 исправлений
Temperature и top_p что-то решают	Все варианты 28-30 из 33, в пределах шума
Команда на скриншоте пробивает защиту	По трейсам это путаница "желание или дефект". Три правки промпта: 50-55 из 70 против 59, откат
mini на черновике не хуже	Метрики те же, в текстах 4 ошибки из 33

Пользователи и следующий шаг

Команда на проде с 21.09

1 человек от бизнеса, 1 РМ, 3 разработчика.

На 23.09: 7 учеток, 3 проекта, 8 замечаний. Из них 7 из моего прогона 20.09. После 21.09 одно новое замечание и 4 решения РМ.

Активность пока низкая.

Что поменяли: уведомления о замечаниях в колокольчике, ADR 016.

Что дальше

- Пилот: один раунд от 30 строк без параллельного Excel
- Собрать отзывы команды об интерфейсе
- OCR сканов ТЗ через vision
- Поиск на корпусе от 100 фрагментов

\$0,0033 за замечание на golden, около \$0,0048 на бою. 300 замечаний и 150 ретестов в месяц: от \$1,10 до \$1,55.

remark-round.up.railway.app

github.com/Dunenbetov/remark-round

MCP-сервер и Skill

- `search_spec`
- `get_round_remarks`
- `apply_human_verdict`
- `submit_retest_evidence`

Инструмента "закрыть" нет. Проект зашит в токен, фильтр в SQL.

Skill uat-triage: один файл для графа, MCP-prompt и Claude Code. Без Skill 6 ответов из 33 меняют класс.

На кадре: Skill подгрузился сам, `search_spec` нашел §2.1 Primary, на вопрос не по ТЗ ответ "Опоры нет". Решение оставлено человеку.

```
› заказчик пишет, что кнопка не синяя - баг или хотелка?

• Skill(remarkround:uat-triage)
  ↳ Successfully loaded skill

• Проверю опору в пакете документов проекта.
  Called remarkround

• Разбор

Опора есть – ТЗ §2.1 Primary: «Primary-кнопка – синяя #0B5FFF».

Класс: defect_candidate – если речь про primary-кнопку (Сохранить/Войти/оплата) и на скрине она не синяя #0B5FFF, это расхождение с ТЗ, а не пожелание.

Что нужно уточнить у заказчика перед подачей вердикта человеку:
- Скрин обязателен – без кадра визуальный дефект не утверждаю.
- Какая именно кнопка? Если secondary (§2.2) – она и не должна быть синей, тогда это change_request_candidate или недоразумение.
- Есть нюанс: §5 в ТЗ конфликтует с §2.1 («серая допустима, если так привычнее бухгалтерии»). Если исполнитель сошлётся на §5 – класс unspecified (конфликт в бумагах), решает человек.

Точку ставит владелец через apply_human_verdict – не я.

* Worked for 20s

› поищи в ТЗ проекта: сколько стоит доставка пиццы на Марс?

Called remarkround

• Опоры нет. В пакете документов этого проекта (ТЗ Клиентского кабинета v1.4) нет ни слова про доставку пиццы, тем более на Марс – лучший фрагмент дал близость 0.25 при пороге 0.45.

Раздел выдумывать не буду. Если это была проверка на галлюцинации – она пройдена: система честно возвращает «нет опоры», и я это уважаю.

* Crunched for 7s
```

Трейсинг, защита, ограничения

Трейсинг

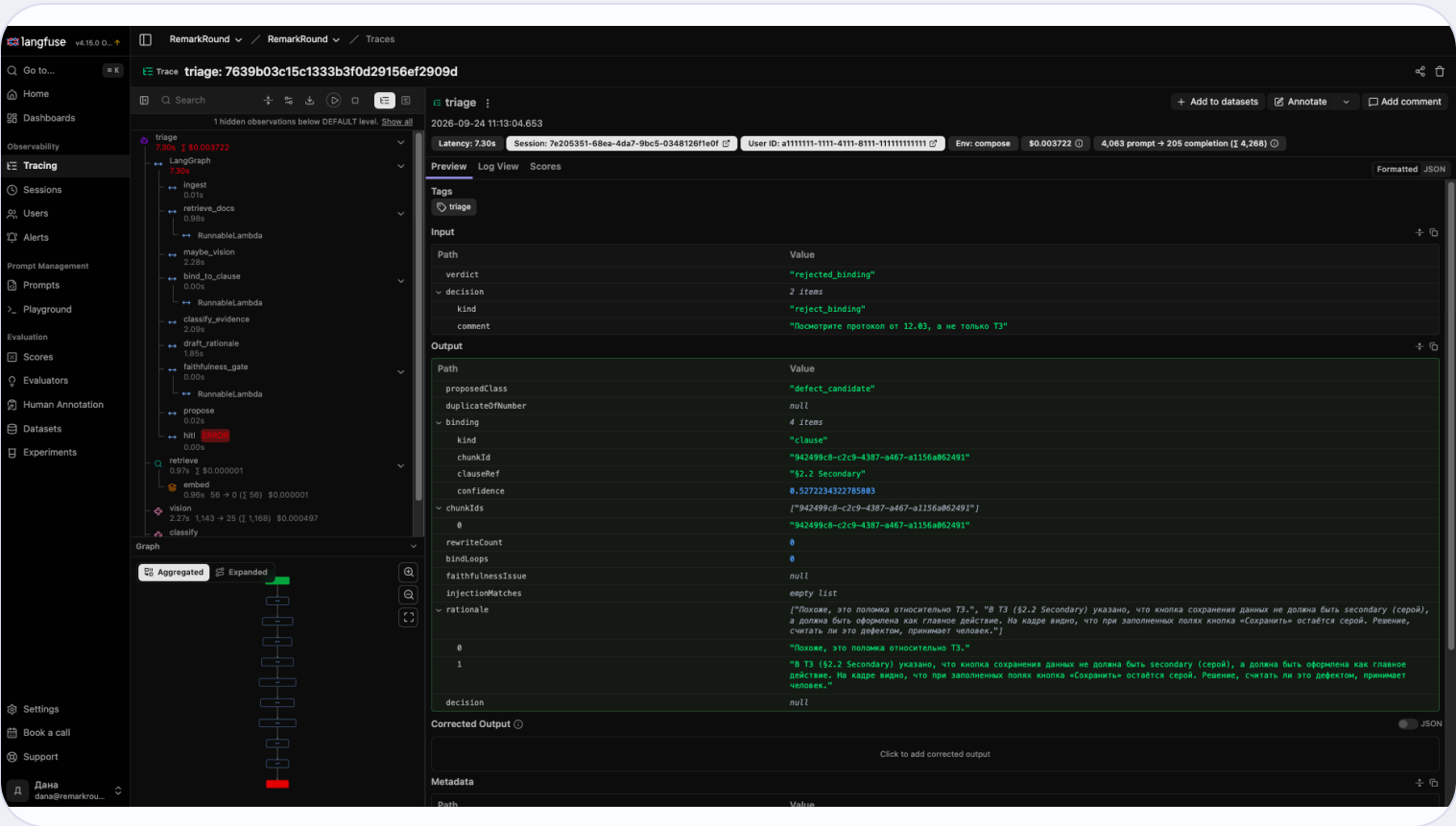
Langfuse Cloud. Один прогон, один трейс: `traceld = sha256(runld)`. Счет на 17 % ниже нашего прайса: кэш промпта.

Защита

Детектор инъекций по тексту, 15 правил. Черновик проверяет код. Фильтр проекта в SQL. Закрывает только заказчик.

Ограничения

Один инстанс API. Кадры разного размера не сравниваются. Детектор не читает кадр. Golden синтетический.



Полный список ограничений с замерами: docs/EVALS.md, раздел 13. Соответствие требованиям курса: таблица в README.